

Traps

2026-03-23

1s2026

Gabriela Bittencourt, João Pedro Leôncio and Vinícius Peixoto

Contents



1	Introduction	3
1.a	Motivation	4
1.b	Idea: polling	5
2	Traps	8
2.a	Definition	9
2.b	Example	10
2.c	Trap numbers	11
2.d	Trap handlers	15
3	Trap handling in our mini kernel	21
3.a	Sv39 canonical addresses	22
3.b	Lower vs. higher half kernels	25
3.c	Lower half kernels	26

1 Introduction

Motivation



Certain events in a system may need immediate attention:

- Devices
 - User input, such as typing on a keyboard, moving a mouse, etc.
 - GPU finishes a job (e.g. rendering the next frame of your videogame)
 - Network interface card (NIC) receives a new network packet
 - And many more
- Software error conditions (e.g. someone messed up):
 - Invalid instructions
 - Page faults (invalid access to pages)

Question: how to deal with those events?

Idea: polling



- Most hardware devices are exposed to software through **memory-mapped registers**
 - e.g. hardware registers can be accessed by reading/writing to memory addresses
 - For example, the serial device our kernel uses to print to the console:

```
char *uart = (char*)0x10000000;  
void putchar(char c)  
{  
    *uart = c;  
    return;  
}
```

- But also many others, e.g. PCI devices (GPUs, NICs, SATA/NVMe controllers, etc.)

Idea: polling (ii)



Idea: continuously check those memory-mapped registers for any events

- This is called **busy-waiting**

```
#define DEV_DATA_AVAIL  *(u64*)0x20000000;  
#define DEV_DATA        *(u64*)0x20000008;  
u64 dev_read()  
{  
    volatile u64 data_available = DEV_REG_DATA_AVAIL;  
    volatile u64 data = DEV_REG_DATA;  
  
    while (1) {  
        if (data_available != 0)  
            return data;  
    }  
}
```

What is the **problem** with this?

Idea: polling (iii)



What is the **problem** with this?

- We waste a lot of CPU cycles checking if data is available
- What if we need to do other work in the meantime?
- Stop doing work periodically to check?
 - How often should we check without risking missing data because we were too slow?
- This game gets complicated very quickly when you have multiple devices
- And what if we have a **bug** in our driver (e.g. nullptr deref)?
- Crash the whole system?

There is a better solution: **traps**

2 Traps

Definition



- A **trap** is an event that causes the CPU to stop whatever it is doing and perform a **jump** to another code location upon an event
- There are generally two types of events:
 - ▶ **External**: an external device raises an event that requires immediate attention
 - e.g. a character has arrived in our serial device, or a network packet arrived in a NIC, etc.
 - These are commonly called **interrupts**, or **IRQs** (= interrupt requests)
 - ▶ **Internal**: something in the code that the CPU is running raises an event that requires immediate attention
 - e.g. a page fault (accessing invalid pages), but also dividing by zero, executing privileged or invalid instructions, etc
 - These are commonly called **exceptions** or **faults**

How would our serial driver example work with interrupts?

Example



```
struct {
    // buffer for the data
    u64 data[512];
    // elements available to read
    size_t available;
} dev;

#define DEV_DATA *(u64*)0x20000008
void dev_irq()
{
    volatile u64 reg_data = DEV_DATA;
    dev.data[dev.end++] = reg_data;
    dev.data_available = true;
}
```

```
u64 dev_read()
{
    if (dev.available != 0)
        return dev.data[--dev.end];
    else
        return 0;
}
```

Note how we:

- `dev_irq` is called **asynchronously** to read data upon an interrupt
- We don't block when doing `dev_read` anymore
- We use a buffer → no more risk of dropping data

Trap numbers



There are several different kinds of traps. On most architectures, each trap type is identified by a **number**

- For exceptions, there are usually **architecturally defined** codes identifying each exception type
- For interrupts, there are usually **platform defined** numbers identifying each interrupt type
 - ▶ These are called **IRQ numbers** in the Linux world
 - ▶ Generally used for hardware interrupts
 - ▶ Usually correspond to physical hardware pins connected to the CPU/interrupt controller

Trap numbers (ii)

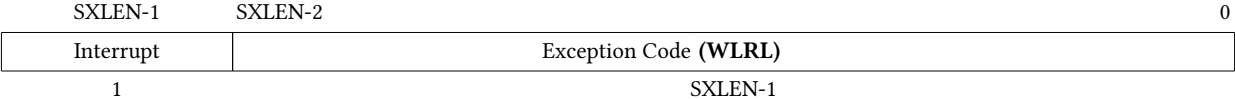


Figure 1: The scause CSR in RISC-V.

Interrupt	Exception Code	Description
1	0	Reserved
1	1	Supervisor software interrupt
1	2-4	Reserved
1	5	Supervisor timer interrupt
1	6-8	Reserved
1	9	Supervisor external interrupt
1	10-12	Reserved
1	13	Counter-overflow interrupt
1	14-15	Reserved
1	≥ 16	Designated for platform use

Figure 2: The scause interrupt codes in RISC-V.

Trap numbers (iii)



0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault
0	8	Environment call from U-mode
0	9	Environment call from S-mode
0	10-11	<i>Reserved</i>
0	12	Instruction page fault
0	13	Load page fault
0	14	<i>Reserved</i>
0	15	Store/AMO page fault
0	16-17	<i>Reserved</i>
0	18	Software check
0	19	Hardware error
0	20-23	<i>Reserved</i>
0	24-31	<i>Designated for custom use</i>
0	32-47	<i>Reserved</i>
0	48-63	<i>Designated for custom use</i>
0	≥ 64	<i>Reserved</i>

Figure 3: The `scause` exception codes in RISC-V.

Trap numbers (iv)



Keep in mind the fundamental difference between **exceptions** and **interrupts**:

- Exceptions are **synchronous**:
 - ▶ Triggered upon executing a code instruction
 - Illegal instruction
 - Load/store/instruction page faults
 - Environment calls (`ecall`)
 - ▶ Identifying codes are architecturally **fixed**
- Interrupts are **asynchronous**:
 - ▶ Triggered by various hardware events that may happen at any time
 - Can have various meanings depending on hardware (serial device has data available, GPU finishes rendering frame, etc.)
 - ▶ IRQ numbers are defined by the **platform** (e.g. each defined by the architecture of the motherboard/system-on-chip)

Trap handlers



A **trap handler** is a function that the CPU jumps to when a trap happens.

There are normally two ways of going about it:

- Have a single function that handles all traps
 - This is what is generally used in RISC-V (AFAIK)
- Have an array of functions that handle different trap types (generally interrupts)
 - This scheme is what is usually called **vectored interrupts** (and the array is the **interrupt vector table**)
 - IRQ 0 triggered → CPU jumps to the address stored in `IVT[0]`, IRQ 1 → `IVT[1]` and so on
 - RISC-V supports this, but we won't use it (for simplicity)

Trap handlers (ii)



A trap handler generally has this structure:

```
void trap_handler()
{
    bool is_irq = trap_is_irq();
    u64 cause = trap_get_cause();

    if (is_irq) {
        handle_irq(cause);
        return;
    }

    switch (cause) {
        case TRAP_PAGE_FAULT:
            handle_page_fault();
            break;
        case TRAP_INVALID_INSTR:
            handle_invalid_instr();
            break;
        ...
    }
}
```


Trap handlers (iii)



There is an additional detail about traps that we have to worry about.

Thinking in terms of assembly, what happens when we **call another function** here?

```
void trap_handler()
{
    bool is_irq = trap_is_irq();
    u64 cause = trap_get_cause();

    if (is_irq)
        handle_irq(cause);
    // ...
}
```

Trap handlers (iv)



Answer: we overwrite (at the very least) `ra` and `a0/a1/a2`!

- Remember that any part of the kernel code can suddenly be interrupted and jump to the trap handler
- We have to make sure the trap handler doesn't overwrite any state (registers) from what was running before
- This is usually achieved by **dumping all registers** onto the stack before entering the trap handler
 - This “block” of register/state dumps on the stack is called a **trap frame**

Trap handlers (v)



```
.globl trap_entry
.align 4
trap_entry:
    addi sp, sp, -144
    sd ra, 0(sp)
    sd sp, 8(sp)
    sd gp, 16(sp)
    # write the rest of the registers...

    call trap_handler

    ld ra, 0(sp)
    ld sp, 8(sp)
    ld gp, 16(sp)
    # reload the rest of the registers...
```

Trap handlers (vi)



Other things to note:

- Your trap handler should be **fast**
 - Otherwise you will hog resources from the rest of the operating system
 - Preferably defer slow/heavy work for later
 - Linux: top vs. bottom halves
- Your trap handler **should never block**
 - e.g. waiting for something to happen for an unknown amount of time
 - This risks having one of the CPUs stuck inside the trap handler forever
- Your regular **kernel code** needs to assume that it can be **preempted** at any time
 - e.g. another higher-priority interrupt can occur during their duration
 - This creates a whole other class of problems that we will cover next class when we talk about **concurrency**

3 Trap handling in our mini kernel

Sv39 canonical addresses



Recall this detail (which you should be familiar with from Lab 01):

Sv39 implementations support a 39-bit virtual address space, divided into 4 KiB pages. [...] Instruction fetch addresses and load and store effective addresses, which are 64 bits, **must have bits 63–39 all equal to bit 38**, or else a page-fault exception will occur.

— The RISC-V Instruction Set Manual

- Why did the RISC-V spec authors bother defining this weird condition?

Sv39 canonical addresses (ii)



Most of you validated this condition by comparing bits directly, e.g.:

```
int vm_map_page(u64 va, phys_addr_t pa, u64 flags) {  
    // ...  
    // validate canonical addresses  
    u64 bit38 = va & (1 << 38);  
    u64 bits63_39 = v >> 39;  
    if (bit38 == 0) return bits63_39 == 0;  
    else return bits63_39 == 0x1fffffff; // 25 bits set to 1  
    // ...  
}
```

This works! (and most of you passed the tests)

But is there another way to look at it?

Notice how when `addr[63:38] == 0`, all addresses **below or equal to**

```
0b000000000000000000000000000000001111111111111111111111111111111111  
    bits[63:38]  
  
= 0x3fffffffffff
```

are valid; similarly, when `addr[63:38] == 1`, all addresses **above or equal to**

[illegible]

are valid. This means we have a “hole” in the middle of our virtual address space:

```
0x3fffffffff < va < 0xffffffffc000000000
=> va is invalid!
```


Lower vs. higher half kernels



- This “hole” in the middle of the VA space is present in most common architectures (x86_64, aarch64, etc).
- This naturally creates a separation of the VA space in two regions:
 - The **lower** half (below the hole)
 - The **higher** half (above the hole)
- A choice that we as kernel developers have to make early on is whether we want to implement a **lower half** or **higher half** kernel

Lower half kernels



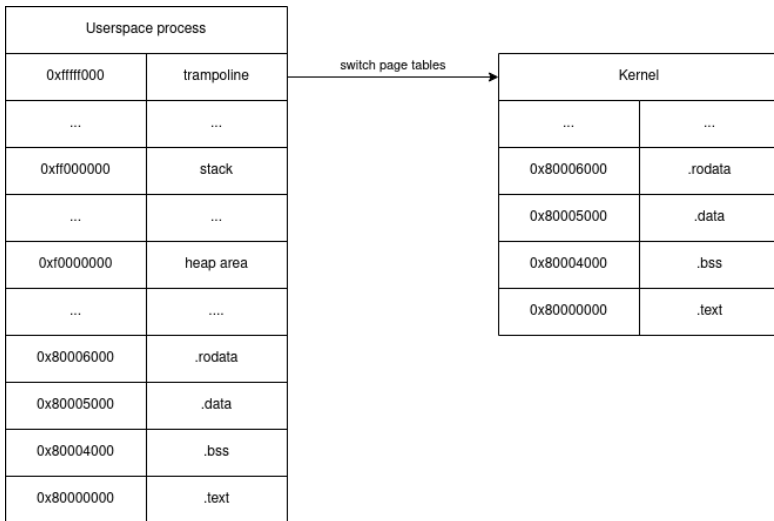
- Example: xv6
- The kernel code and data lives in the lower half
 - e.g., we map `.text`, `.bss`, etc to lower half virtual addresses
- Userspace also lives in the lower half
 - e.g. we map all of userspace to lower half addresses as well, with different page tables
- Pros: generally easier to set up; technically less attack surface since the kernel is not accessible through userspace at all
- Cons: harder for the kernel to access userspace data (e.g. when servicing syscalls); traps require something called a **trampoline**

Lower half kernels (ii)



- Trap handling: upon a trap, switch from userspace to kernel; handle the trap; switch back to userspace
- Map a piece of code called a **trampoline** to some known location in every process (usually the last virtual memory page)
- The role of the trampoline is to perform something called a **context switch** between userspace and kernel:
 - ▶ Save all relevant state (e.g. registers) to the original process stack (such as an userspace process)
 - ▶ Load the relevant state of the new process (such as the kernel process) and switch stacks
 - ▶ **Load the new page tables** and **flush the TLB**

Lower half kernels (iii)



- Problem: context switches are **slow**

Lower half kernels (iv)



- ▶ Reloading page tables + flushing the TLB means we will have lots of cache misses until the cache is warmed up again
- ▶ Doing a context switch every time we need to handle interrupts is very inefficient
- ▶ How can we do improve this?

Lower half kernels (v)

